

Intro: Welcome to CS61B Final!

Name: _____ Your SID: _____

Location: _____ Seat Number: _____

SID of Person to your Left: _____ Right: _____

Formatting:

- $\bigcirc$ indicates only one circle should be filled in. $\square$ indicates more than one box may be filled in. **Please fill in the shape completely.** If you change your response, **erase as completely as possible.**
- Anything you write that you ~~cross out~~ will not be graded.
- You may not use ternary operators, lambdas, streams, or multiple assignment.

Tips:

- This final is worth **100 points**, and there are a lot of problems on this exam. Work through the ones with which you are comfortable first. Do not get overly captivated by interesting design issues or complex corner cases you're not sure about.
- Not all information provided in a problem may be useful, and you may not need all lines.
- We will **not** give credit for solutions that **exceed the number of provided lines** (close braces must lie on their own line) or that **fail to follow any restrictions given in the problem statement**.
- There may be partial credit for incomplete answers. Write as much of the solution as you can, but bear in mind that we may deduct points if your answers are much more complicated than necessary.
- Unless otherwise stated, all given code on this exam compiles. All code has been compiled and executed before printing, but in the unlikely event that we do happen to catch any bugs in the exam, we'll announce a fix.

Write the statement below in the same handwriting you will use on the rest of the exam: "I have neither given nor received help on this exam (or quiz), and have rejected any attempt to cheat. If these answers are not my own work, I may be deducted 9,876,543,210 points on the exam."

Signature: _____

1 Final Curry

(10 Points)

For all questions below, V refers to the set of vertices and E refers to the set of edges in a graph. Unless otherwise stated, assume graphs on this exam are connected and simple.

- (a) If a graph has more than $|V| - 1$ edges, and there is a unique heaviest edge, then this edge cannot be part of any minimum spanning tree (MST).
☐ True ☐ False
- (b) If a graph has more than $|V| - 1$ edges and two edges have the same weight, it must have multiple valid minimum spanning trees.
☐ True ☐ False
- (c) We can get the maximum spanning tree of a graph by negating the edge weights and running any minimum spanning tree algorithm on it.
☐ True ☐ False
- (d) The minimum spanning tree of a graph can be used to find the shortest path between any combination of 2 vertices.
☐ True ☐ False
- (e) MSD Radix Sort works correctly if we sort each individual digit with Heap Sort instead of Counting Sort.
☐ True ☐ False
- (f) LSD Radix Sort works correctly if we sort each individual digit with Heap Sort instead of Counting Sort.
☐ True ☐ False
- (g) You have an array representation of a min-heap. If you reverse the array, it will become a valid representation of a max-heap.
☐ True ☐ False
- (h) A^* may visit more nodes than Dijkstra's to find the shortest path of a destination node even if a heuristic is admissible and consistent.
☐ True ☐ False
- (i) An A^* heuristic that is both admissible and consistent will always have a lower or equal heuristic value than the actual cost to reach the destination vertex for each vertex in the graph.
☐ True ☐ False

2 Next Episode

(13 Points)

Angel can't wait for winter break to start. He is designing a program to help him watch TV shows and starts off by representing each episode that he will watch as a linked-list as shown.

```
public class Episode {
    private String name;
    private Episode next;
    // ... implementation not shown ...
}
```

- (a) Reminiscing on his times in Midterm 1, Angel wants to watch his shows in reverse Faro shuffle order. **Write a function that destructively reorders the list of episodes with the elements of even index together followed by the elements of odd index, and return the reordered list.** The relative order inside both the even and odd groups should remain as it was in the input.

For example, on a linked list with nodes that are 0-indexed,

ep0 → ep1 → ep2 → ep3 → ep4 → ep5 will result in the order ep0 → ep2 → ep4 → ep1 → ep3 → ep5

```
public class Episode {
    private String name;
    private Episode next;
    // ... implementation not shown ...

    public Episode reverseFaroShuffle(Episode head) {
        if (head == null) return null;
        Episode even = head;
        Episode odd = head.next;
        Episode oddHead = odd;

        while (_____ 1 _____ && odd.next != null) {

            even.next = _____ 2 _____;

            even = _____ 3 _____;

            odd.next = _____ 4 _____;

            odd = _____ 5 _____;
        }

        even.next = _____ 6 _____;
        return head;
    }
}
```

- (b) For each of the following lines, write the result of each statement: Write "CE" if a compiler error is raised on that line, "RE" if a runtime error occurs on that line, "OK" if the line runs properly but doesn't print anything, and the printed result if the line runs properly and prints something. If a line errors, assume that the program continues to run as if that line did not exist. Blank lines will receive no credit.

```

public class Rating {
    public void judgeRating(Rating r) {
        System.out.println("True");
    }
    public void judgeRating(ProfessionalRating r) {
        System.out.println("Very True");
    }
}
public class ProfessionalRating extends Rating {
    public void judgeRating(Rating r) {
        System.out.println("Indeed Bogus");
    }
    public void judgeRating(ProfessionalRating r) {
        System.out.println("Indeed Factual");
    }
}
public class AudienceRating extends Rating {
    public void judgeRating(Rating r) {
        System.out.println("So valid");
    }
    public void judgeRating(ProfessionalRating r) {
        System.out.println("Not Valid");
    }
}

```

Rating rating = new Rating();	// _____
	1
AudienceRating audRating = new AudienceRating();	// _____
	2
ProfessionalRating profRating = new ProfessionalRating();	// _____
	3
Rating ratingProf = new ProfessionalRating();	// _____
	4
Rating[] ratings = new Rating[10];	// _____
	5
ratings[1].judgeRating(ratingProf);	// _____
	6
rating.judgeRating(audRating);	// _____
	7
((ProfessionalRating) rating).judgeRating(rating);	// _____
	8
rating.judgeRating((ProfessionalRating) ratingProf);	// _____
	9

3 Asymptotics

(12 Points)

Give the asymptotic runtimes for the functions below in Θ notation. Here are some common arithmetic rules that may be of use:

- $\sqrt{n} = n^{(0.5)}$
- $\log_a(b) - \log_a(c) = \log_a(b/c)$
- $\log_a(b) + \log_a(c) = \log_a(b \cdot c)$
- $\log_a(b^n) = n \log_a(b)$
- $\log_a(b) = \log_c(b) / \log_c(a)$
- $a^{\log_a(b)} = b$

(a)

```
public static void q1(int N) {
    for (int i = 1; i < N; i += 1) {
        for (int j = 1; j < N; j *= 2) { System.out.println("ben"); }
    }
}
```

$\Theta($)

(b)

```
public static void q2(int N) {
    if (N <= 1) { return; }
    if ((new Random()).nextInt() % 2 == 0) {
        for (int i = 0; i < N / 2; i += 1) { System.out.println("beben"); }
        q2(N - 1);
    } else {
        q2(N - 4);
    }
}
```

Best Case

$\Theta($)

Worst Case

$\Theta($)

```

(c) double q3(double N) {
    if (N <= 10) {
        return 1;
    }
    double C = N;
    while (C > 1) {
        System.out.println("aniruth");
        C = C / 2;
    }
    return q3(Math.sqrt(N));
}

```

$$\Theta(\quad)$$

```

(d) double q4(double N) {
    if (N <= 10) { return 1; }
    double C = N;
    while (C > 1) {
        System.out.println("anianiruth");
        C = C / 2;
    }
    return q4(Math.sqrt(N)) + q4(Math.sqrt(N));
}

```

(1) What is the height of the recursive tree in terms of N ? We will define this as h .

$$h = \Theta(\quad)$$

(2) What is the asymptotic runtime in terms of N and h ?

$$\Theta(\quad)$$

4 Evil Sets

(7 Points)

(a) (i)

Index	0	1	2	3	4	5
Value	-1	-1	-1	-1	-1	-1

(ii)

Index	0	1	2	3	4	5
Value	-5	0	3	0	0	0

(iii)

Index	0	1	2	3	4	5
Value	-6	5	0	0	0	1

(iv)

Index	0	1	2	3	4	5
Value	0	-6	1	0	0	0

Which of the following are valid states of a weighted quick-union (**without** path compression)? Select all that apply.

☐ (i)

☐ (ii)

☐ (iii)

☐ (iv)

☐ None of the above

- (b) Eddie creates a weighted quick-union, and this is the state of the underlying array.

Index	0	1	2	3	4	5	6	7	8	9	10	11
Value	-3	-2	0	7	9	1	2	-5	3	-2	8	3

1. The indices 0 and 9 are connected. ☐ True ☐ False
2. The indices 10 and 11 are connected. ☐ True ☐ False
3. The set with element 6 and the set with element 5 are the same size. ☐ True ☐ False

Eddie realizes that the height of the disjoint sets is growing too fast. **He decides to change the implementation to a WQUPC** (Weighted Quick Union with Path Compression) using the same array representation above. For the rest of this question, assume that tie breaking is done by choosing the smaller number to become the root.

4. What is the state of the underlying array after calling `union(10, 5)` starting from the same array representation above?

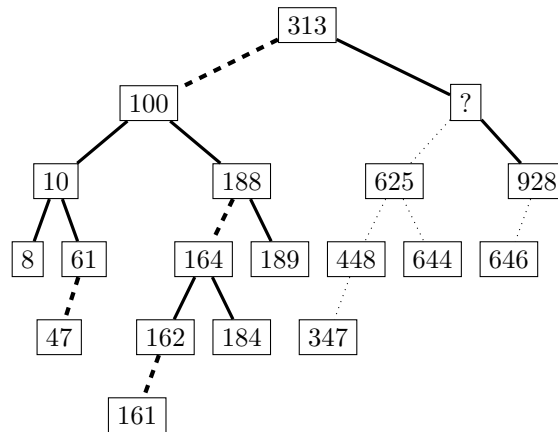
Index	0	1	2	3	4	5	6	7	8	9	10	11
Value												

5. Suppose we restarted with the disjoint set that we had at the beginning of the part (b), and we're still using a WQUPC. What is the minimum number of `union()` operations that we need to do to fully connect the disjoint set **and** reduce the height of the tree to 1?

5 Bushy Trees

(12 Points)

- (a) Consider the following incomplete LLRB Tree. **Thick solid links** are black edges, **thick dashed links** are red edges, and **thin dotted links** are unknown edges. (e.g. 100-10 is black, 61-47 is red, and 625-448 is unknown)



1. What must be the value of the node? Your answer must be an integer.

2. Select all node(s) above that **must** have a red edge leading to their parent.

☐ 347 ☐ 448 ☐ 625 ☐ 644 ☐ 646 ☐ None of the above

3. Suppose we insert the value 88 into the LLRB. How many of the following operations are performed?

rotateLeft

rotateRight

colorFlip

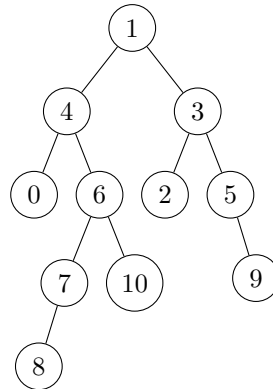
4. Suppose we insert the value 170 into the original LLRB (without 88). How many of the following operations are performed?

rotateLeft

rotateRight

colorFlip

- (b) The cats, Fish and Chips, are selling cookies to their neighbors. Their neighborhood can be represented as a tree of houses, with each house having a number representing how many cookies the cats can sell to that house. Both cats start selling on the root node labeled 1. Assume `sellCookies` sells the maximum number of cookies to the given house.



```

1. void fish(House house) {
    if (house == null) { return; }
    sellCookies(house);
    fish(house.left);
    fish(house.right);
}

```

What is the total number of cookies Fish sold after calling `sellCookies` on **six** houses?

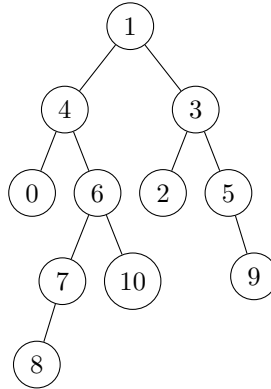
```

2. void chips(House house) {
    if (house == null) { return; }
    chips(house.right);
    chips(house.left);
    sellCookies(house);
}

```

What is the total number of cookies Chips sold after calling `sellCookies` on **six** houses?

The tree is placed again for your convenience.



- (c) Fish discovers that they can sell cookies to a neighbor multiple times by calling `sellCookies(house)` on the same house. In order to take advantage of this, Fish writes the function `fishDoubleDip`. Fish sells the following amounts of cookies during the first twelve calls to `sellCookies`:

1, 4, 0, 0, 6, 7, 8, 8, 7, 10, 10, 6

Fill in the definition of `fishDoubleDip` below.

```

void fishDoubleDip(House house) {
    if (house == null) { return; }
    // _____ line 1 _____
    // _____ line 2 _____
    // _____ line 3 _____
    // _____ line 4 _____
}

1. // _____ line 1 _____
   ○ sellCookies(house)    ○ fishDoubleDip(house.left)    ○ fishDoubleDip(house.right)

2. // _____ line 2 _____
   ○ sellCookies(house)    ○ fishDoubleDip(house.left)    ○ fishDoubleDip(house.right)

3. // _____ line 3 _____
   ○ sellCookies(house)    ○ fishDoubleDip(house.left)    ○ fishDoubleDip(house.right)

4. // _____ line 4 _____
   ○ sellCookies(house)    ○ fishDoubleDip(house.left)    ○ fishDoubleDip(house.right)

```

6 Snack Scores

(5 Points)

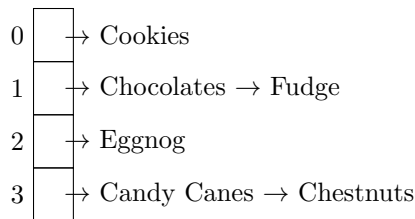
Vanessa wants to rate snacks. She constructs the following `Snack` class then stores snacks in a `HashSet`.

```
public class Snack {
    public String name;
    public int rating;
    // Constructor is omitted

    @Override
    public boolean equals(Object obj) {
        if (obj instanceof Snack s) {
            return this.name.equals(s.name) && this.rating == s.rating;
        }
        return false;
    }

    @Override
    public int hashCode() { return rating; }
}
```

For this question, assume we use `Math.floorMod(hashCode, buckets.length)` to find the corresponding bucket index. After adding all snacks, the `HashSet` looks like this:



- (a) Assume we add a new snack "Peppermint Bark" with `hashCode = -1`. Which bucket will it end up in assuming that it does not trigger a resize?

☐ 0 ☐ 1 ☐ 2 ☐ 3

- (b) When the holidays are over, Vanessa changes the rating of "Candy Canes" without updating the `HashSet`. Later, when she tries to find "Candy Canes", she gets an error saying her `HashSet` does not contain "Candy Canes". What are the possible values that the new rating of "Candy Canes" to be?

☐ -19 ☐ -13 ☐ 0 ☐ 10 ☐ 15 ☐ 19 ☐ None of the above

- (c) The `HashSet` is resized by doubling the size of the backing array to 8. Which `hashCodes` will end up in a different bucket index than before the resize?

☐ -9 ☐ -8 ☐ -3 ☐ 0 ☐ 5 ☐ 11 ☐ None of the above

7 Priority Queue

(10 Points)

- (a) An array of 7 distinct integers is constructed into a max-heap. Which of the following is a possible ordering for the array? The 0th index is denoted with a $-$ as it is empty in our implementation of a heap.

☐ $[-, 7, 6, 1, 4, 5, 2, 3]$

☐ $[-, 1, 7, 6, 5, 4, 3, 2]$

☐ $[-, 7, 3, 6, 2, 1, 5, 4]$

☐ $[-, 7, 4, 6, 5, 3, 2, 1]$

☐ $[-, 7, 6, 3, 1, 2, 4, 5]$

☐ $[-, 7, 6, 5, 4, 3, 2, 1]$

- (b) David has a basket of peaches, and wants to get the perfect peach, which is the peach with the middle weight. In order to do this, David decides to create a `MiddleHeap`, which is a Priority Queue that returns the middlemost element (instead of the greatest/least element like a `MaxHeap`/`MinHeap`). Formally, the middle of $2n + 1$ items is the n th smallest item (0 indexed), and the middle of $2n$ items is the $n - 1$ th smallest item. For example, the middle of $\{1, 2, 3\}$ is 2, and the middle of $\{2, 3, 4, 5\}$ is 3. Implement the methods of the `MiddleHeap` class.

```
public class MiddleHeap<Integer> {

    private MaxHeap<Integer> left = new MaxHeap<Integer>();
    private MinHeap<Integer> right = new MinHeap<Integer>();

    public int findMiddle() {
        if (left.isEmpty()) {
            throw new NoSuchElementException("Priority queue underflow");
        }
        return left.findMax();
    }

    public int deleteMiddle() {
        if (left.isEmpty()) {
            throw new NoSuchElementException("Priority queue underflow");
        }

        int k = left.deleteMax();
        if (left.size() < right.size()) {

            left.insert(_____);
        }
        return k;
    }
}
```

```

public void insert(int k) {
    if (left.isEmpty()) {

        _____2;

        return;
    }

    if (k < left.findMax()) {

        _____3;

    } else {

        _____4;

    }

    if (left.size() > _____5) {

        _____6.insert(_____7);

    }

    if (left.size() < _____8) {

        _____9.insert(_____10);

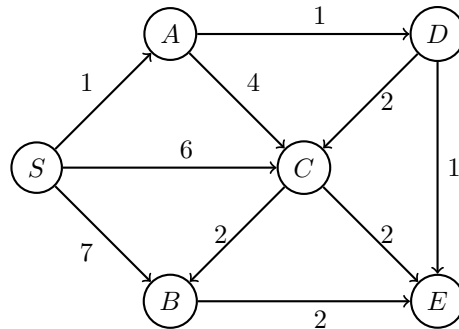
    }
}
}

```

8 Lots of Graphite

(6 Points)

Consider the graph below. When running Dijkstra's and A*, assume that we will tiebreak in alphabetical order.



(a) Run Dijkstra's algorithm from S. What is the order that vertices get visited?

- ☐ S, A, D, C, E, B
- ☐ S, A, C, B, D, E
- ☐ S, A, D, E, C, B
- ☐ S, A, B, C, D, E
- ☐ S, A, D, C, B, E
- ☐ None of the above

(b) Select all valid topological sorts:

- ☐ S, A, D, C, E, B
- ☐ S, A, C, B, D, E
- ☐ S, A, D, E, C, B
- ☐ S, A, B, C, D, E
- ☐ S, A, D, C, B, E
- ☐ None of the above

Node	Value
S	7
A	6
B	2
C	4
D	6
E	0

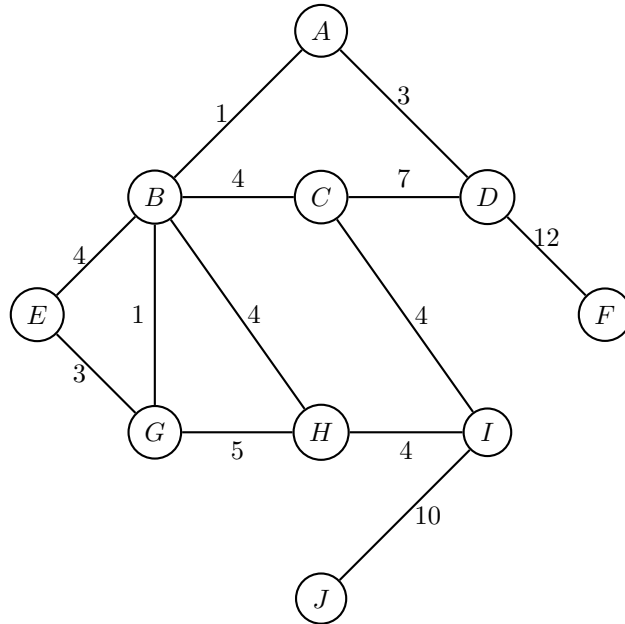
(c) Run A* from S to E, given the above heuristic. What is the order that vertices get visited?

- ☐ S, A, D, E, B
- ☐ S, A, B, E
- ☐ S, A, D, E, C, B
- ☐ S, A, D, C, B, E
- ☐ S, A, D, E
- ☐ None of the above

9 World Wide Web

(5 Points)

Portia is a spider trying to build a path between all of the trees, shown as nodes in the graph below. The edge weights represent the amount of silk she needs to build a path between the two trees.



- (a) Run Prim's starting from node C, tiebreaking by picking alphabetically first edges (i.e. BG before BE, BE before CD, CD instead of DC'). Write the sequence of edges added to the MST, in the order that they are added. The first edge has been given to you.

BC								
----	--	--	--	--	--	--	--	--

- (b) We run Kruskal's algorithm with an unknown tiebreaking scheme. Which of the following edges could possibly have been the 5th edge added to the MST?

☐ AB ☐ AD ☐ BC ☐ BE ☐ BG ☐ BH ☐ CD ☐ CI ☐ DF ☐ GH ☐ HI ☐ IJ

- (c) Portia eats some high-tensile-strength bugs, and can now make silk that's 2 times as strong as before; as a result all edges in the graph now have $1/2$ their original weight. Every MST in the original graph is still an MST of the new graph.

☐ True ☐ False

10 Sorting Out the Situation

(7 Points)

- (a) Eric implemented a new sorting algorithm below. You may assume that this sorting algorithm works as intended and sorts values in ascending order.

```
public static void swap(int[] arr, int indexA, int indexB) {
    int tempVal = arr[indexA];
    arr[indexA] = arr[indexB];
    arr[indexB] = tempVal;
}

public static void beben(int[] arr) {
    for (int i = arr.length - 1; i >= 0; i--) {
        for (int j = 0; j < i; j++) {
            if (arr[j] >= arr[j + 1]) { swap(arr, j, j+1); }
        }
    }
}
```

1. This sort sorts values in place. ☐ True ☐ False
2. This sort is stable. ☐ True ☐ False
3. Insertion sort makes strictly fewer swaps than **beben**. ☐ Always ☐ Sometimes ☐ Never
4. Selection sort makes strictly fewer swaps than **beben**. ☐ Always ☐ Sometimes ☐ Never

- (b) Select the best sort amongst the provided options for the situation.

1. You are assigned with the task of sorting 1,000,000 random distinct numbers and you don't want to use additional memory.

☐ Quick Sort with Hoare Partitioning
☐ Merge Sort
☐ Insertion Sort

☐ Selection Sort
☐ Counting Sort
2. You are given a sorted array and call `swap(arr, (new Random()).nextInt(arr.length), (new Random()).nextInt(arr.length))` once.

☐ Quick Sort with Hoare Partitioning
☐ Merge Sort
☐ Insertion Sort

☐ Selection Sort
☐ Counting Sort
3. You are making a review page where users can submit reviews as integers between 1 and 5. Reviews are stored by their timestamp. You want to make a feature that sorts reviews based on their rating but keeps the order the review is submitted if the ratings are the same.

☐ Quick Sort with Hoare Partitioning
☐ Merge Sort
☐ Insertion Sort

☐ Selection Sort
☐ Counting Sort
4. You accidentally sort a list of numbers that is already fully sorted.

☐ Quick Sort with Hoare Partitioning
☐ Merge Sort
☐ Insertion Sort

☐ Selection Sort
☐ Counting Sort

11 Belt Of Heapolyta

(13 Points)

Heracles has been tasked with yet another labour: to find the Magic Belt of the queen of the Amazons, Heapolyta. Heapolyta has granted permission for Heracles to take the belt, but only if he can sort Heapolyta's collection of both magic and nonmagic belts. Heapolyta has informed Heracles of the following:

- All belts have different weights. You may compare two belts to determine which is heavier.
- There is exactly one magic belt. The Magic Belt has been enchanted to “lie” about its weight; it will appear heavier than any belt that's heavier than it, and lighter than any belt that's lighter than it.
- The two lightest belts and the two heaviest belts are not magic. (Otherwise, it would be impossible to correctly sort the belts)

More specifically, the Belt class is defined as follows:

```
class Belt implements Comparable<Belt> {
    private int weight;
    private boolean isMagic;

    public int compareTo(Belt belt) {
        int delta = this.weight - belt.weight;
        if (this.isMagic) { delta *= -1; }
        if (belt.isMagic) { delta *= -1; }
        return delta;
    }
}
```

For the following questions, we will write “X” to denote a nonmagic belt with weight X, and “X*” to denote a magic belt. For example, “3*” represents a weight 3 Magic Belt, and “4” represents a weight 4 Nonmagic Belt. If we compared belt 3* to 4, we would be told that 3* is heavier.

(a) Assume that initially, the belts are laid out in the following pattern:

3*, 6, 2, 5, 1, 0, 4, 7

Select the results of the following sorts out of the possible answers. Some answers may be used more than once and each sort has only one correct answer:

- | | | | |
|----------------------------|----|----------------------|---|
| a. 3*, 0, 1, 2, 4, 5, 6, 7 | 1. | <input type="text"/> | Insertion Sort |
| b. 0, 1, 2, 4, 5, 6, 7, 3* | | | |
| c. 6, 3*, 0, 1, 2, 4, 5, 7 | 2. | <input type="text"/> | Quick Sort (3-Way Partitioning with 0th index pivot) |
| d. 0, 3*, 1, 2, 4, 5, 6, 7 | | | |
| e. 4, 5, 6, 7, 3*, 0, 1, 2 | 3. | <input type="text"/> | Quick Sort (Hoare Partitioning with 0th index pivot) |
| f. 0, 1, 6, 3*, 2, 4, 5, 7 | | | |
| g. 0, 1, 2, 3*, 4, 5, 6, 7 | | | |
| h. Does not terminate | 4. | <input type="text"/> | beben Sort, found on the previous page, assuming it has been modified to accept arbitrary comparables (ex. it uses <code>(arr[j].compareTo(arr[j+1]) >= 0)</code> in the if clause). |
| i. None of the above | | | |

- (b) Find an ordering of 9 belts of weight 1 - 9 such that after running Quick Sort with 3-way partitioning using the first element as a pivot, at least 16 inversions exist in the resulting list among the Non-Magic belts. That is, once the Magic Belt is removed, there are 16 inversions in the list of Non-Magic belts. An inversion will be counted if $\text{arr}[i]$ is greater than $\text{arr}[j]$ where i is lesser than j .

--	--	--	--	--	--	--	--	--

- (c) You are given a list of N belts of unique weights, exactly one of which is magic. Write a program to sort the belts, placing the Magic Belt in the index as if the belt was not magic. Your algorithm must run in $O(N \log N)$ time. You have access to the following black-box functions:
- `findMagicBelt`, which receives as input a `List<Belt>` subject to the constraints of Heapolyta's collection of belts (i.e. exactly one magic belt, at least two nonmagic belts lighter and two nonmagic belts heavier than the magic belt), and returns the index corresponding to the magic belt. You may assume that this runs in $\Theta(N)$ time (see Bonus Question).
 - `heapSort`, which receives as input a `List<E extends Comparable<E>>` and applies an in-place Heap Sort with bottom-up heapification (as described in lecture) to the list. You may assume that this runs in $\Theta(N \log N)$ time.

You may not create new belts (in other words, comparisons can only be done between existing belts), and you may not access the instance variables `weight` or `isMagic` of each belt directly. All data structures and algorithms covered in class are under your disposal. Your answer may be an English description or pseudocode. **Your answer must fit in the box provided, and we will not grade anything outside the box.**

--

Nothing on this page is worth any points.

61Bonus Question

(0 Points)

Describe an algorithm `findMagicBelt` that receives as input a `List<Belt>` containing exactly one magic belt, and returns the index of the magic belt in $\Theta(N)$ time. You may not create new belts (in other words, comparisons can only be done between existing belts).

Feedback

(0 Points)

Leave any feedback, comments, concerns, or drawings below!

61B Final Clarifications

9a: In the example listed, "BE" should be alphabetically before "BG"

10a 3: "Insersion" should be "Insertion"